Український державний університет науки і технологій
ННІ «ПРИДНІПРОВСЬКА ДЕРЖАВНА АКАДЕМІЯ БУДІВНИЦТВА ТА АРХІТЕКТУРИ»

Кафедра комп’ютерних наук, інформаційних технологій та прикладної математики

ЛАБОРАТОРНА РОБОТА №3

КЛАСИФІКАЦІЯ ОБРАЗІВ ЗА ДОПОМОГОЮ ОДНОШАРОВОГО ПЕРСЕПТРОНА, З НОРМУВАННЯМ ВХІДНИХ ДАНИХ

Виконав студент КН-25
Лобов Є.С.

Перевірив старший викладач
Плахтій Є.Г.

м. Дніпро 2026

ТЕОРЕТИЧНА ЧАСТИНА

Для розв’язання будь-якої прикладної задачі за допомогою штучної нейронної мережі необхідно виконати п’ять етапів: підготувати дані для навчання, сформувати архітектуру ШНМ, ініціалізувати мережу, виконати навчання, протестувати навчену ШНМ.

Для вирішення задач класифікації вхідних векторів, які відносяться до класу лінійно роздільних, використовуються одношарові нейронні мережі, які називають персептрони.

Формування архітектури мережі.

Для створення ШНМ, яка має архітектуру одношарового персептрона, в пакеті Neural Network Toolbox використовується функція **perceptron**:

> net = perceptron (hardlimitTF, perceptronLF).

Вхідні аргументи:

hardlimitTF – активаційна функция (за замовчуванням **'hardlim'** – порогова);

perceptronLF – функція навчання (за замовчуванням **'learnp'**)

Вихідні аргументи:

net – об’єкт класу Neural Network з архітектурою одношарового персептрона. Кількість нейронів персептрона визначається автоматично в процесі навчання, виходячи з того, що одношаровий персептрон, який містить m нейронів, здатен розділити вхідний простір на 2m класів. Значення ваг та зміщень персептрона за замовчуванням ініціалізуються за допомогою функції **init**, яка встановлює нульові значення.

Існує інший спосіб сформувати одношаровий персептрон за допомогою функції **newp**:

> net = newp (X, T, s, hardlimitTF, perceptronLF).

Вхідні аргументи:

X – масив розмірністю R×Q, який містить вхідні вектори, де R – кількість елементів, з яких складений вхідний вектор, Q – кількість векторів;

T – масив розмірністю S×Q, який містить вихідні вектори, де S – кількість
елементів, з яких складений вихідний вектор;

s – кількість нейронів.

Приклад. Сформувати персептрон для розподілу трьох 2-елементних векторів [0; 0], [1; 1] та [2; 2] на два класи: 0 та 1:

> X = [0 1 2; 0 1 2];

> T = [0 1 1];

Після того як сформована архітектура мережі, необхідно задати початкові значення ваг та зміщень. Для цього можна використовувати:

а) функцію **init**, яка присвоює вагам та зміщенням нейронів персептрона нульові значення:

> net = init (net);

б) функцію **rands**, яка генерує випадкові значення масивів ваг та зміщень:

> net.IW{1, 1} = rands (s, R) – генерує матрицю розмірністю s×R, елементи якої приймають випадкові значення з інтервала [–1; 1], та присвоює отримані значення матриці ваг нейронів першого шару персептрона;

> net.b{1} = rands (s) – генерує вектор розмірністю s×1, елементи якого приймають випадкові значення з інтервала [–1; 1], та присвоює отримані значення вектору зміщень нейронів першого шару персептрона;

в) встановити бажані значення ваг та зміщень.

Приклад. Присвоїти вагам та зміщенню одношарового персептрона, який містить один нейрон, наступні значення: w1 = −3 , w2 = 2 , w0 = 5.

> net.IW{1, 1} = [–3; 2];

> net.b{1} = 5.

Підготовка даних для навчання

Вхідні вектори X можуть мати два формати: **cell array та double array.** Формат **cell array** відповідає *послідовному* способу подачі даних на вхід мережі, який можна інтерпретувати в такий спосіб. На вхід мережі подається послідовність з n вхідних сигналів, тобто на першому кроці – перший, на другому – наступний, і так далі. У відповідь мережа генерує послідовність виходів, яка також містить n елементів. Результат був би той самий, якби було створено n однакових мереж, і на кожну мережу було подано один із вхідних векторів.

Подання вхідних даних як числового масиву в форматі **double** відповідає *пакетному* способу представлення даних, в результаті чого на виході мережі генерується відповідний масив.

Приклад. Сформувати чотири види вхідних даних: один вектор-стовпець з двох елементів; група (пакет) з двох трьохелементних векторів; група з трьох двоелементних векторів; послідовність з трьох двоелементних векторів:

1) X1 = [1; 2];
2) X2 = [1 4; 2 5; 3 6];
3) X3 = [1 3 5; 2 4 6];
4) X4 = {[1; 2] [3; 4] [5; 6]}.

Навчання мережі.

Щоб підготувати нейронну мережу для вирішення конкретної задачі, необхідно налаштувати її параметри, тобто виконати навчання. У пакеті програм Neural Network Toolbox реалізовано два способи навчання: *послідовний* та *пакетний* (груповий).

Для організації послідовного навчання використовується функція **adapt**, яка коригує значення ваг та зміщень за результатами одноразової подачі на вхід мережі масиву вхідних даних:

> net_adapt = adapt (net, X, T).

Вхідні аргументи:

net – ім’я нейронної мережі до навчання;

X – масив вхідних векторів;

T – масив бажаних значень вихідних векторів (масив цілей).

Вихідні аргументи:

net_adapt – ім’я нейронної мережі після навчання.

Приклад. Реалізувати процедуру навчання нейронної мережі, використовуючи наступну послідовність навчаючих шаблонів

$$\{X_1=\begin{bmatrix}-1\\-1\end{bmatrix}, T_1=0\}, \{X_2=\begin{bmatrix}-1\\1\end{bmatrix}, T_2=0\}, \{X_3=\begin{bmatrix}1\\-1\end{bmatrix}, T_3=1\}, \{X_4=\begin{bmatrix}1\\1\end{bmatrix}, T_4=1\}$$

> X = {[–1; –1] [–1; 1] [1; –1] [1; 1]}

> T = {0 0 1 1};

> net_1 = adapt (net, X, T).

Необхідно зауважити, що функція **adapt** виконує одноразове коригування параметрів мережі, тобто не гарантує, що після цього мережа

виконає правильну класифікацію нового вхідного вектора. Для реалізації ітеративної процедури навчання необхідно організувати цикл, на кожному кроці якого застосовується процедура **adapt** та перевіряється, чи був виконаний критерій зупинки навчання (наприклад, поточний вектор похибок мережі містить нульові значення).

Для автоматичної реалізації ітеративної процедури навчання використовується функція train, синтаксис якої є аналогічним. При застосуванні функції train у відповідне вікно на моніторі виводиться схематичне зображення побудованої мережі та в реальному часі відображаються результати навчання: кількість виконаних епох навчання та значення помилки на виході мережі (за замовчуванням – середня абсолютна помилка). Якщо задача є нерозв'язною, тоді процес навчання буде зупинений тільки після виконання встановленої кількості епох (за замовчуванням 1000).Користувач може в будь-який момент зупинити процес навчання за допомогою кнопки «Stop Training», а також вивести на монітор графік залежності величини похибки від кількості епох навчання (кнопка «Performance»). Для зручності відображення результатів користувач може змінити логарифмічний масштаб по осі ординат (обраний за замовчуванням) на лінійний, вибравши в меню «Edit» пункт «Axes Properties».

Тестування (моделювання) мережі

Після закінчення процесу навчання необхідно перевірити, наскільки успішно навчена ШНМ здатна вирішити конкретну задачу. Це можна зробити двома способами: за допомогою моделювання та шляхом графічного представлення результатів навчання.

Моделювання складається з подачі на вхід мережі тестових даних та порівняння результатів, які генерує мережа, з очікуваними. Для моделювання використовується функція **sim**:

> Y = sim (net_train, X).

Вхідні аргументи:

net_train – об'єкт класу Neural Network, навчання якого вже завершено;

X – масив, який містить вхідні вектори, призначені для тестування мережі.

Вихідні аргументи:

Y – масив, який містить вихідні вектори, що є результатом роботи ШНМ.

Для графічного відображення результатів навчання та моделювання використовуються функції **plotvec**, **plotpv**, **plotpc.**

Функція plotvec (X, color, Marker) відображає в двовимірному або в тривимірному просторі вектори-стовпці масиву X у вигляді маркерів, тип яких визначає строкова змінна Marker, а колір кожного маркера визначається елементами вектора-рядка color.

Приклад. Відобразити кружками різних кольорів чотири двоелементних вхідних вектора:

> X = [1.0 1.3 1.7 2.0; 0 –1.0 0.5 –1.5];

> plotvec (X, [1 2 3 4], 'o').

Функція plotpv (X, T, v) відображає в двовимірному або в тривимірному просторі вектори-стовпці масиву входів X розміру R×Q у вигляді маркерів різного типу відповідно до значень масиву цілей T розміру S×Q; вектор-рядок v дозволяє задати границі відображуваного простору у форматі [x_min x_max y_min y_max].

Приклад. Відобразити маркерами різного типу чотири двоелементних вектора, які містить масив X, що належать двом класам:

> T = [0 0 1 1];

> plotpv (X, T, [0.5 2.5 –2 1]).

Результат представлено на рис. 1.1, а.

Функція plotpc (net.IW{1,1}, net.b{1}) будує лінію (в двовимірному просторі) або площину (в тривимірному), яка розділяє простір у відповідності до значень матриці ваг net.IW{1,1} та вектора зміщень net.b{1}

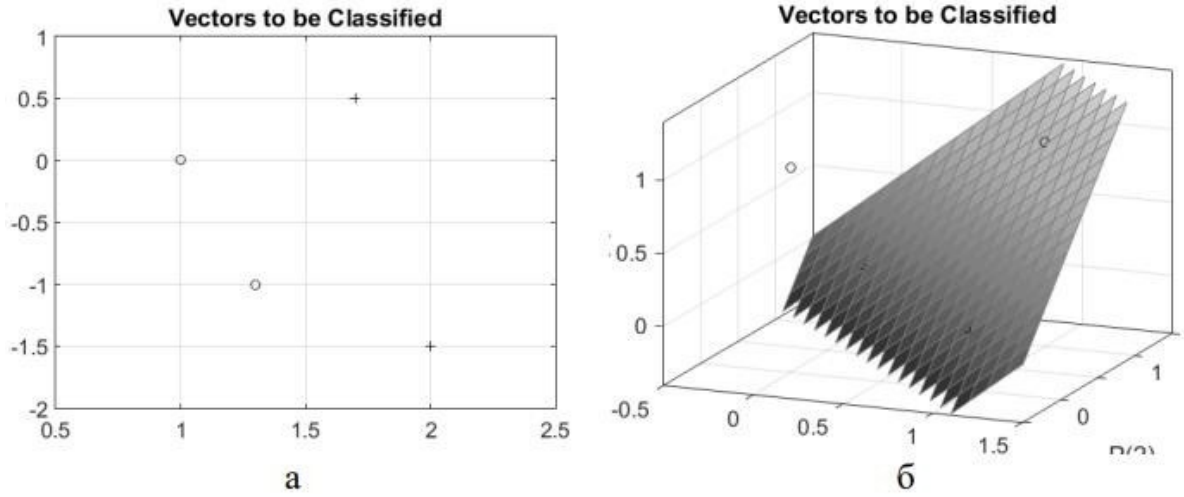


Рисунок 1.1

навченої нейронної мережі. Функцію **plotpc** доцільно використовувати разом з функцією plotpv, яка відображає розміщення в просторі вхідних векторів.

Приклад. Відобразити розміщення в тривимірному просторі чотирьох вхідних векторів, що містяться в масиві X та належать двом класам, які визначаються масивом цілей T, а також площину, відповідно до якої одношаровий персептрон, навчання якого проводилося за допомогою масивів X та T, розділяє простір на два класи:

```
> X = [0 0 1 1; 0 1 0 1; 1 0 0 1];
> T = [0 0 0 1];
> plotpv (X, T);
> hold on;
> plotpc (net.IW{1, 1}, net.b{1});
```

Результат представлено на рис. 1.1, б.

Процедура навчання персептрона має особливості. Одна з них полягає в наступному. Якщо значення елементів деякого вхідного вектора, що використовується при навчанні, значно відрізняються одне від одного, або від значень інших векторів, то для навчання може знадобитися чимало часу. Це обумовлено тим, що алгоритм навчання пов’язаний з додаванням (або відніманням) вхідного вектора до поточного вектору ваг. Таким чином, наявність вхідного вектора з дуже великими або малими елементами призводить до того, що настройка параметрів персептрона вимагає дуже великої кількості ітерацій.

Врівноважити вплив великих чи малих компонент можна за допомогою масштабування вхідних даних. Необхідно нормувати дані так, щоб вплив будь-якого вхідного вектора мав приблизно рівний внесок. Отже, використання нормованого правила навчання персептрона значно скорочує кількість циклів навчання, коли зустрічаються викиди вхідних векторів.

Для обчислення величини корекції матриці ваг та вектора зміщень персептрона можуть застосовуватися дві функції: стандартна **'learnp'** та нормована **'learnpn'**. Нормована функція може бути вказана в якості аргументу при створенні персептрона за допомогою функції perceptron або **newp** (за замовчуванням використовується стандартна функція).

Крім величини норми вхідних векторів, процедура навчання персептрона є дуже чутливою до розміру навчаючої вибірки та порядку пред’явлення

навчальних шаблонів. На практиці число навчальних шаблонів експоненціально зростає із зростанням кількості параметрів мережі, що настроюються. У книзі [4] наведено загальні рекомендації щодо розміру набору навчальних шаблонів. Рекомендується формувати навчальну вибірку так, щоб виконувалася наступна нерівність

Однак,

$$N > \frac{W}{s}$$

де N – кількість навчальних шаблонів, W – кількість параметрів мережі, що настроюються, ε – відсоток помилок, допустимий в ході тестування мережі.

В процесі навчання рекомендується переміжати вхідні образи, що належать до різних класів, а також «перетасовувати» (shuffling) навчальні шаблони.Один з можливих підходів полягає в тому, щоб на кожній ітерації алгоритму випадковим чином вибирати образ навчальної вибірки (тобто під час роботи алгоритму образи навчальної вибірки перебираються в випадковому порядку, можливо, неодноразово).

Якщо застосування нормованої функції '**learnpn**' не дозволяє успішно завершити процедуру навчання персептрона, рекомендується масштабувати вхідні дані таким чином, щоб значення векторів належали інтервалу [0, 1] або [–1, 1]. Для цього необхідно знайти максимальне значення кожної з ознак розподілити значення о знаки кожного вектора на відповідні максимальні, де m – розмірність вектора входів: $x_i^{norm} = x_i/x_i^{max}$

ПРАКТИЧНА ЧАСТИНА

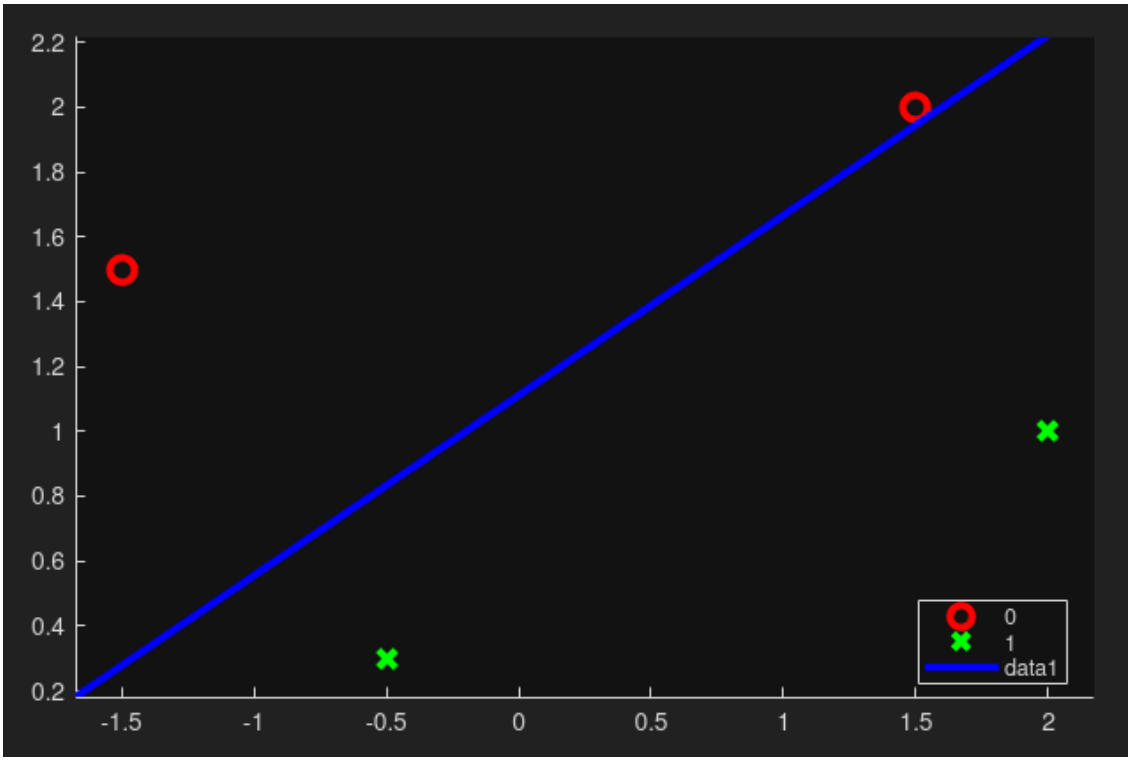
ВАРІАНТ 3

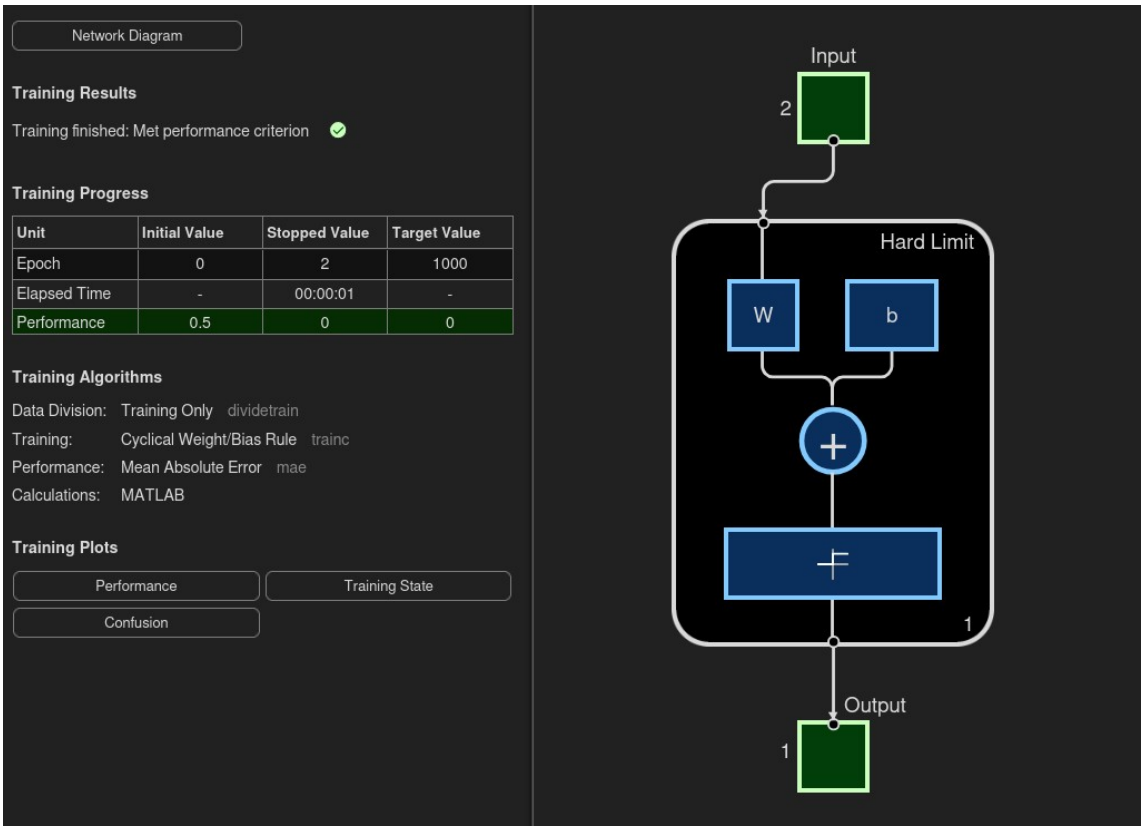
1 ЧАСТИНА

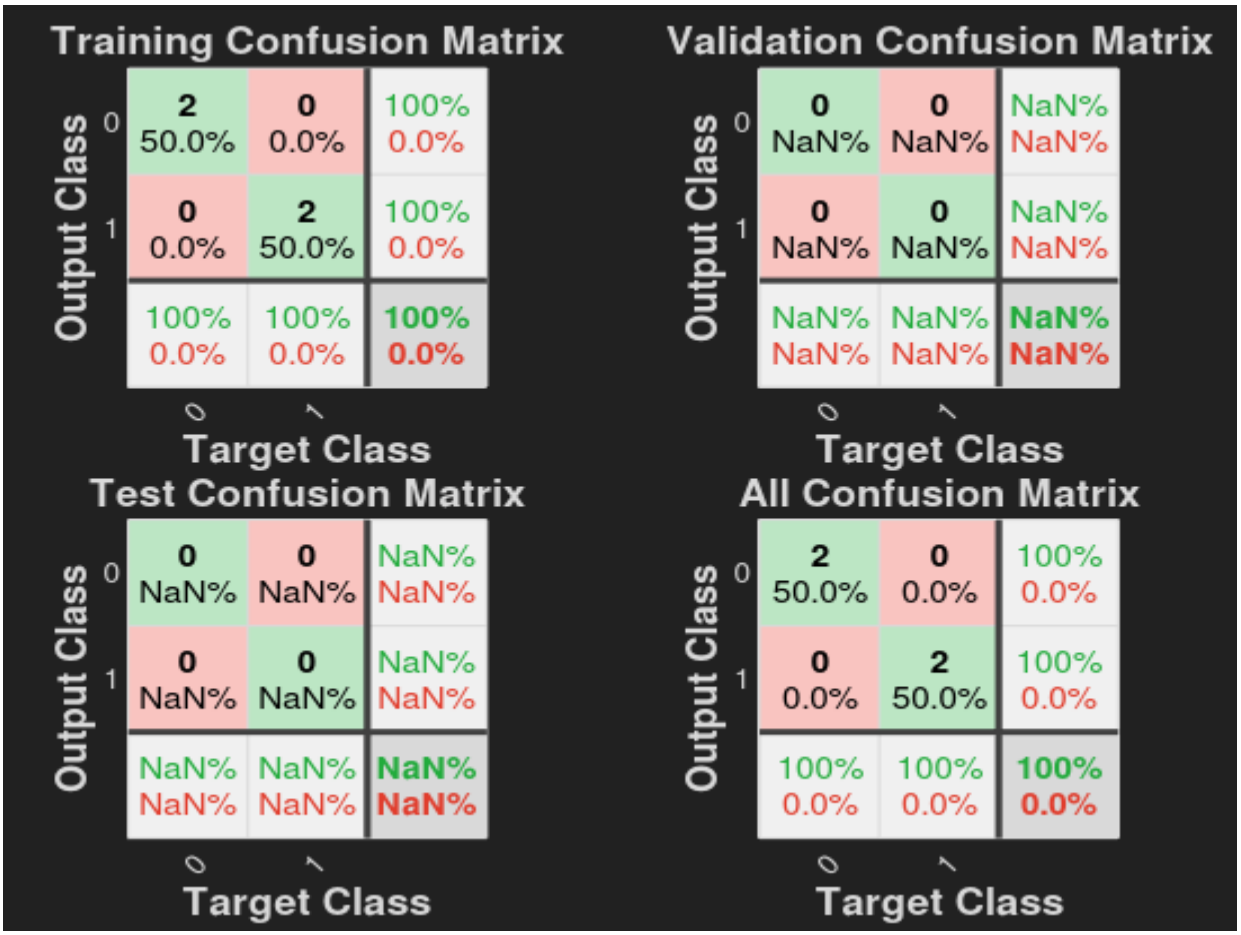
1.Вибрати архітектуру та сформувати штучну нейронну мережу для вирішення задачі класифікації двоелементних образів на два класи.

```
X = [-1.5 1.5 -0.5 2.0;
     1.5 2.0  0.3 1.0];

T = [0 0 1 1];


net = perceptron;
net.numInputs = 1;
net.inputs{1}.size = 2;
net.layers{1}.size = 1;
net.layers{1}.transferFcn = 'hardlim';
net = train(net, X, T);

gscatter(X(1,:), X(2,:), T, ['r', 'g'], 'ox', 10);
hold on;
plotpc(net.IW{1}, net.b{1});
h = findobj(gca, 'Type', 'line');
set(h, 'LineWidth', 3);
```

2. Побудувати графік, який зображує простір входів, вхідні вектори та лінію, яка відображує, як навчена нейронна мережа розділяє простір входівна дві області.


2.2
2
1.8
1.6
1.4
1.2
1
0.8
0.6
0.4
0.2
-1.5
-1
-0.5
0
0.5
1
1.5
2
0
1
data1

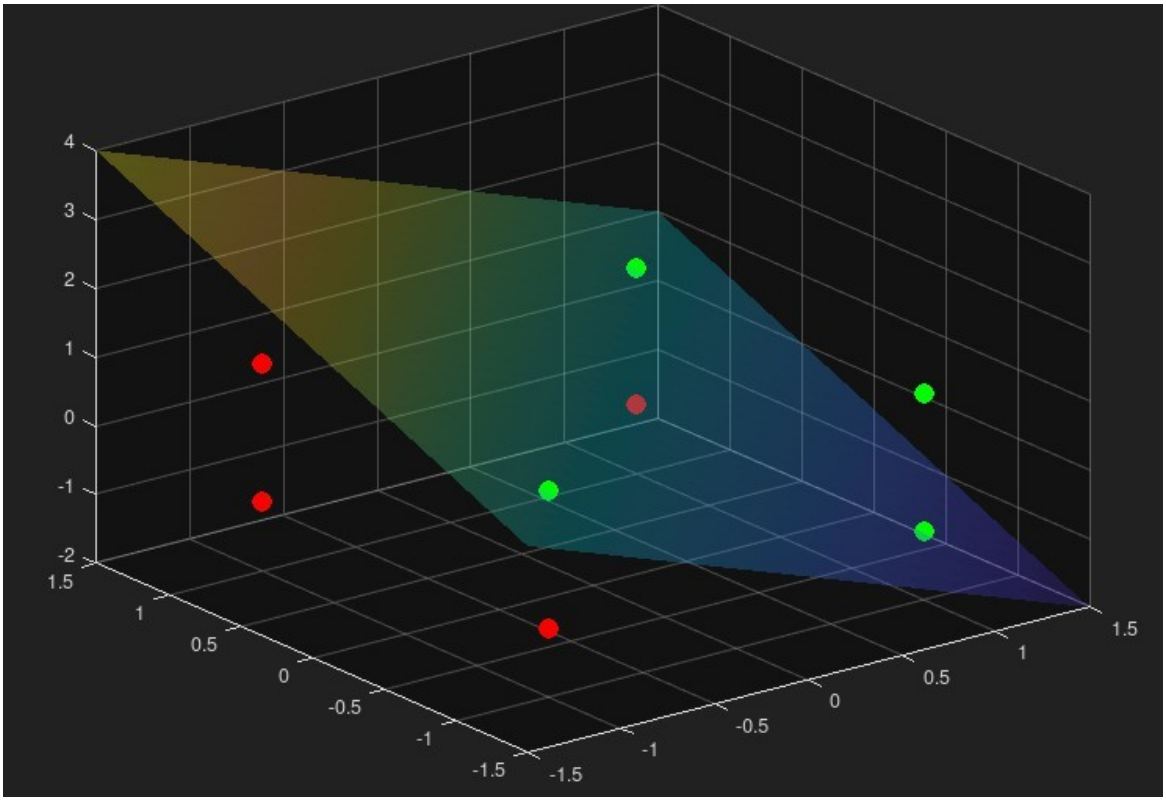

Network Diagram
Training Results
Training finished: Met performance criterion
Training Progress
Unit | Initial Value | Stopped Value | Target Value
Epoch | 0 | 2 | 1000
Elapsed Time | - | 00:00:01 | -
Performance | 0.5 | 0 | 0
Training Algorithms
Data Division: Training Only dividetrain
Training: Cyclical Weight/Bias Rule trainc
Performance: Mean Absolute Error mae
Calculations: MATLAB
Training Plots
Performance
Training State
Confusion
Input
2
Hard Limit
W
b
1
Output
1



3.Вибрати архітектуру та сформувати штучну нейронну мережу для виріше ння задачі класифікації трьохелементних образів на два класи.

```
X = [-1.0  1.0 -1.0  1.0 -1.0  1.0 -1.0  1.0;
     -1.0 -1.0  1.0  1.0 -1.0 -1.0  1.0  1.0;
     -1.0 -1.0 -1.0 -1.0  1.0  1.0  1.0  1.0];

T = [0 1 0 0 1 1 0 1];

net = perceptron;
net.numInputs = 1;
net.inputs{1}.size = 3;
net.layers{1}.size = 1;
net.layers{1}.transferFcn = 'hardlim';


net_train = train(net, X, T);

W = net_train.IW{1,1};
b = net_train.b{1};


figure;
hold on; grid on;
view(3);


scatter3(X(1, T==0), X(2, T==0), X(3, T==0), 100, 'r', 'filled', 'DisplayName', 'Клас 0');
scatter3(X(1, T==1), X(2, T==1), X(3, T==1), 100, 'g', 'filled', 'DisplayName', 'Клас 1');

[x_grid, y_grid] = meshgrid(-1.5:0.1:1.5, -1.5:0.1:1.5);


z_grid = -(W(1)*x_grid + W(2)*y_grid + b) / W(3);
surf(x_grid, y_grid, z_grid, 'FaceAlpha', 0.3, 'EdgeColor', 'none', 'DisplayName', 'Розділяюча площина');
```

4.Побудувати графік, який зображує простір входів, вхідні вектори та площину, яка відображує, як навчена нейронна мережа розділяє простір входів на дві області.



Network Diagram

Training Results

Training finished: Met performance criterion

Training Progress

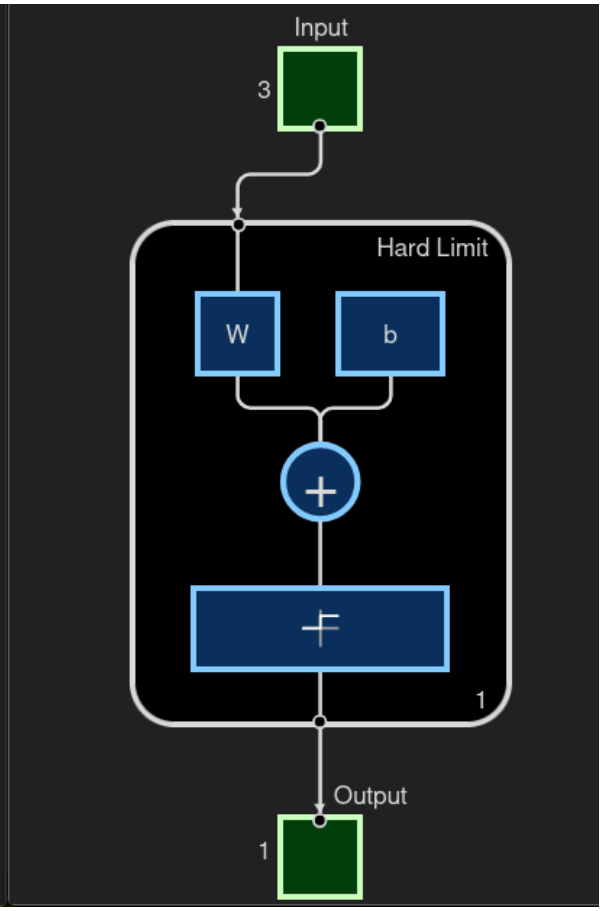
Unit	Initial Value	Stopped Value	Target Value
Epoch	0	1	1000
Elapsed Time	-	00:00:00	-
Performance	0.5	0	0

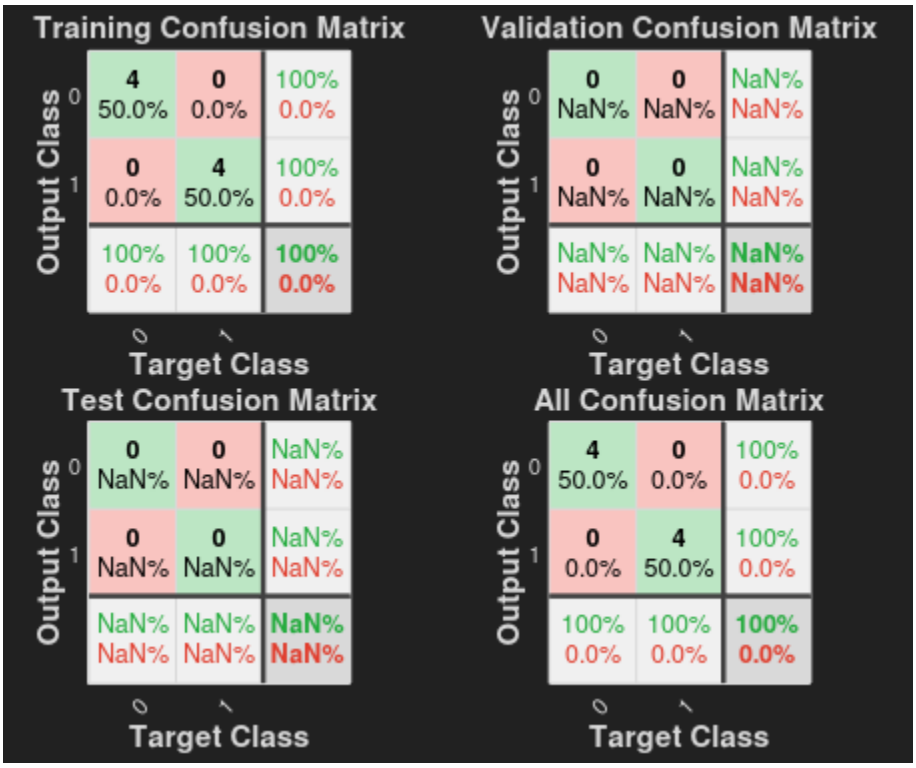
Training Algorithms

Data Division: Training Only dividetrain
Training: Cyclical Weight/Bias Rule trainc
Performance: Mean Absolute Error mae
Calculations: MATLAB

Training Plots

Performance | Training State
Confusion

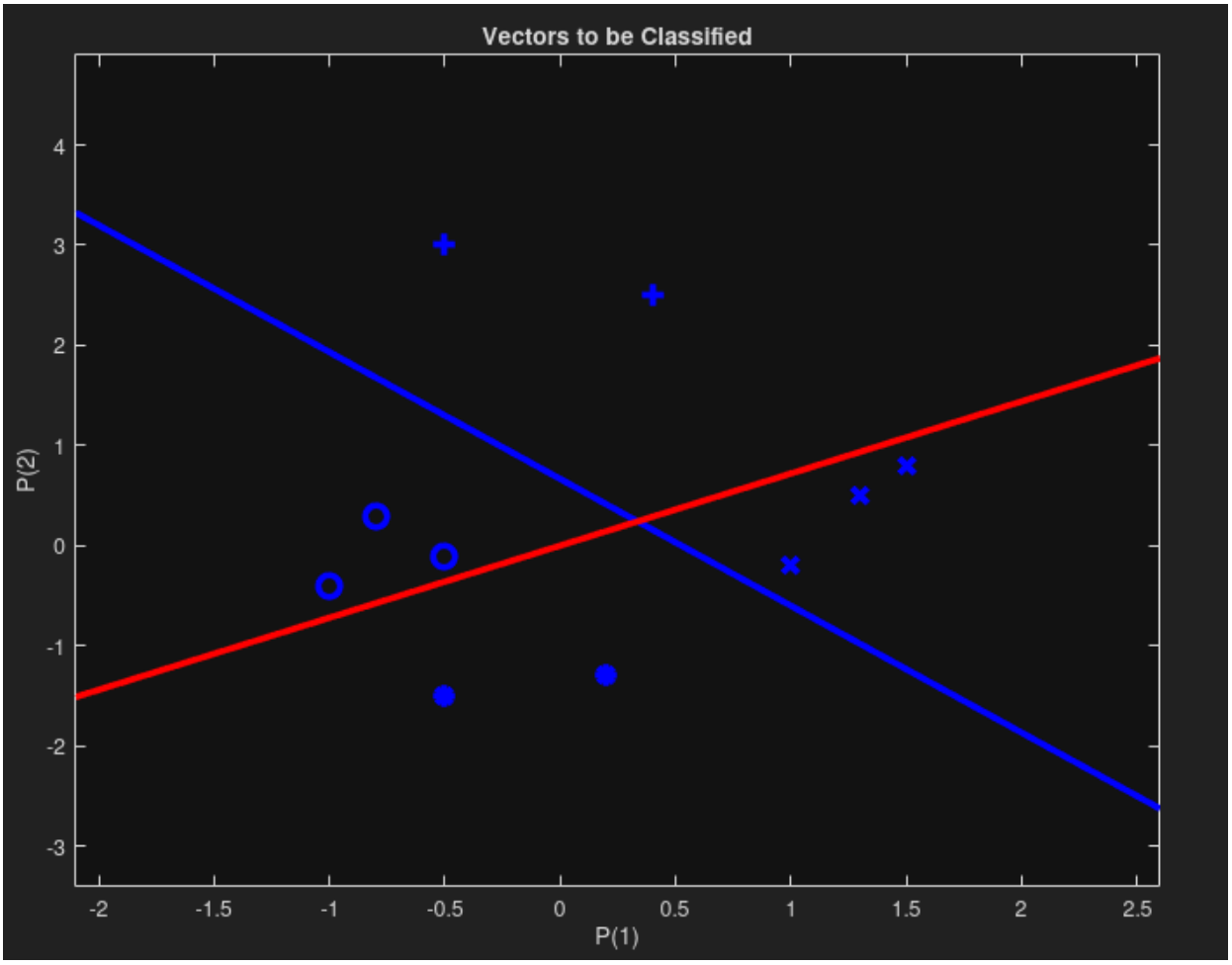




5.Вибрати архітектуру та сформувати штучну нейронну мережу для вирішення задачі класифікації двохелементних образів на чотири класи.

```
X = [-0.5 0.4 -0.8 -1.0 -0.5 1.3 1.0 1.5 -0.5 0.2;
      3.0 2.5  0.3 -0.4 -0.1 0.5 -0.2 0.8 -1.5 -1.3];

T = [1 1 0 0 0 1 1 1 0 0;
     0 0 0 0 0 1 1 1 1 1];

net = perceptron;
net_train = train(net, X, T);


plotpv(X, T);
h_points = findobj(gca, 'Type', 'line');
set(h_points, 'MarkerSize', 10, 'LineWidth', 3);

hold on;


plotpc(net_train.IW{1,1}, net_train.b{1});
h = findobj(gca, 'Type', 'line');
set(h, 'LineWidth', 3)
```

6.Побудувати графік, який зображує простір входів, вхідні вектори та лінії, яка відображує, як навчена нейронна мережа розділяє простір входів на чотири області.



Network Diagram

Training Results

Training finished: Met performance criterion

Training Progress

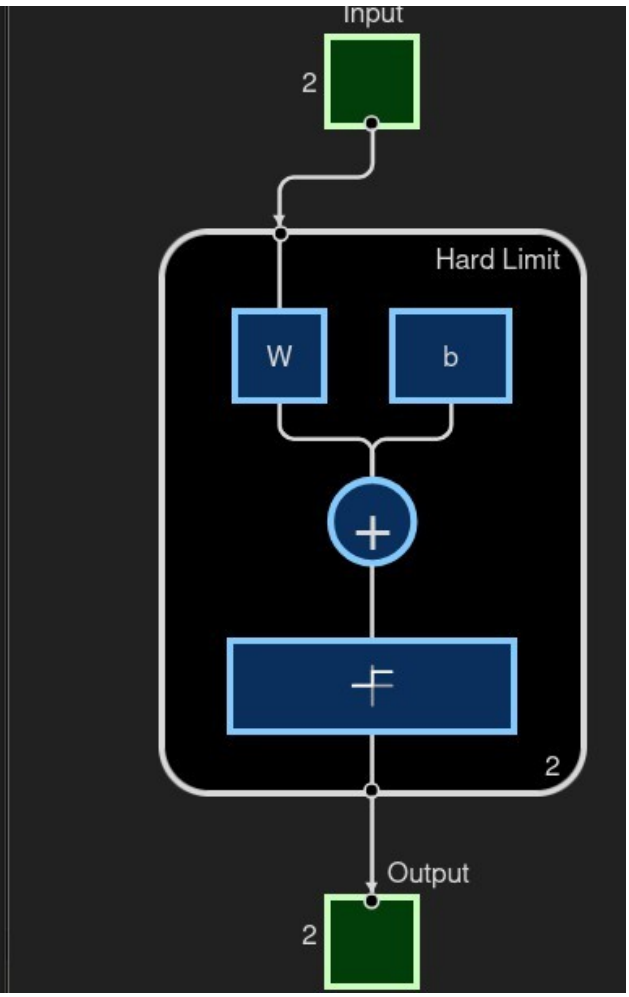
Unit	Initial Value	Stopped Value	Target Value
Epoch	0	1	1000
Elapsed Time	-	00:00:00	-
Performance	0.5	0	0

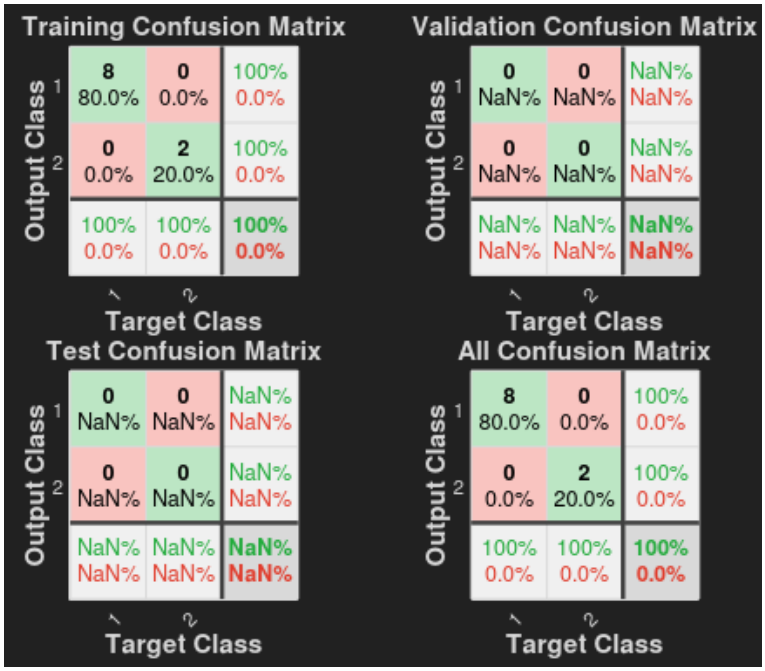
Training Algorithms

Data Division: Training Only dividetrain
Training: Cyclical Weight/Bias Rule trainc
Performance: Mean Absolute Error mae
Calculations: MATLAB

Training Plots

Performance | Training State | Confusion





2 ЧАСТИНА

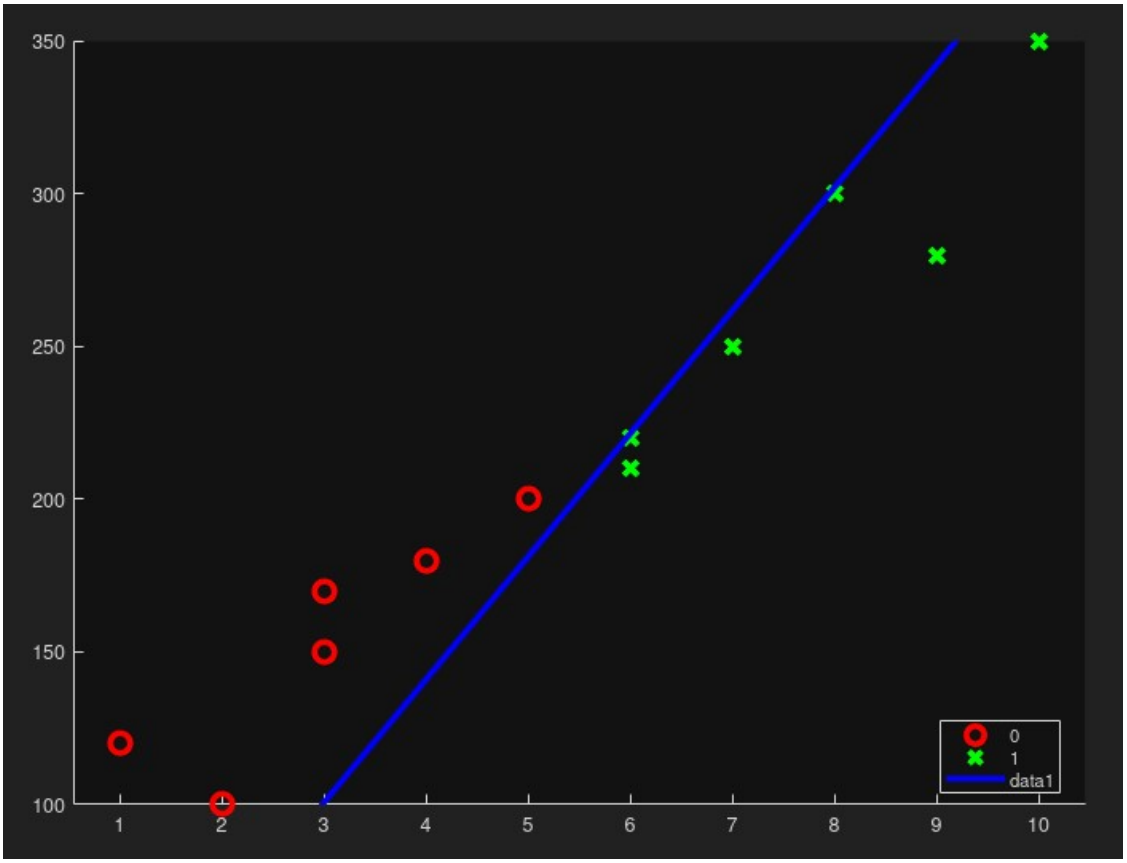
1. Вибрати архітектуру та сформувати штучну нейронну мережу для вирішення задачі класифікації.

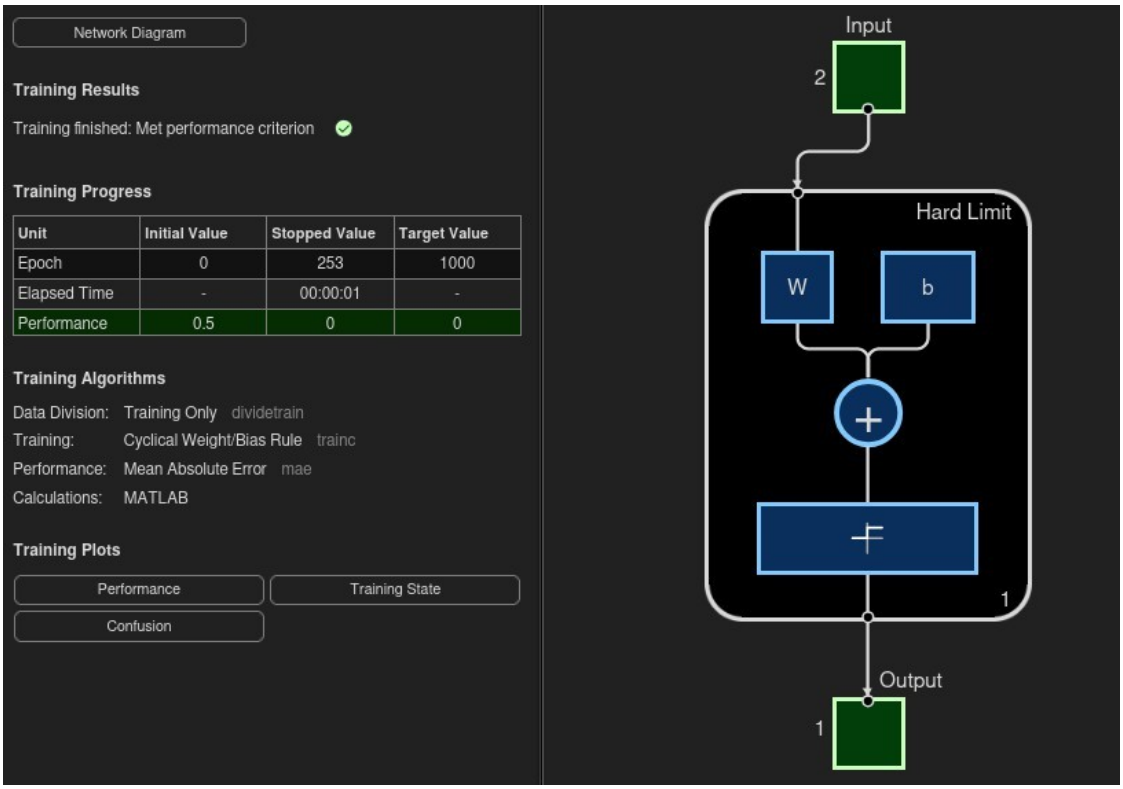
- сформувати навчальну вибірку, яка містить достатню кількість шаблонів, кожен з яких включає вхідний вектор та відповідний вихідний вектор;
- сформувати персептрон відповідної розмірності, вказавши в якості функції навчання '**learnpn**';
- реалізувати процес навчання персептрона за допомогою функції **adapt** або **train**;
- шляхом моделювання перевірити результат роботи навченої мережі.

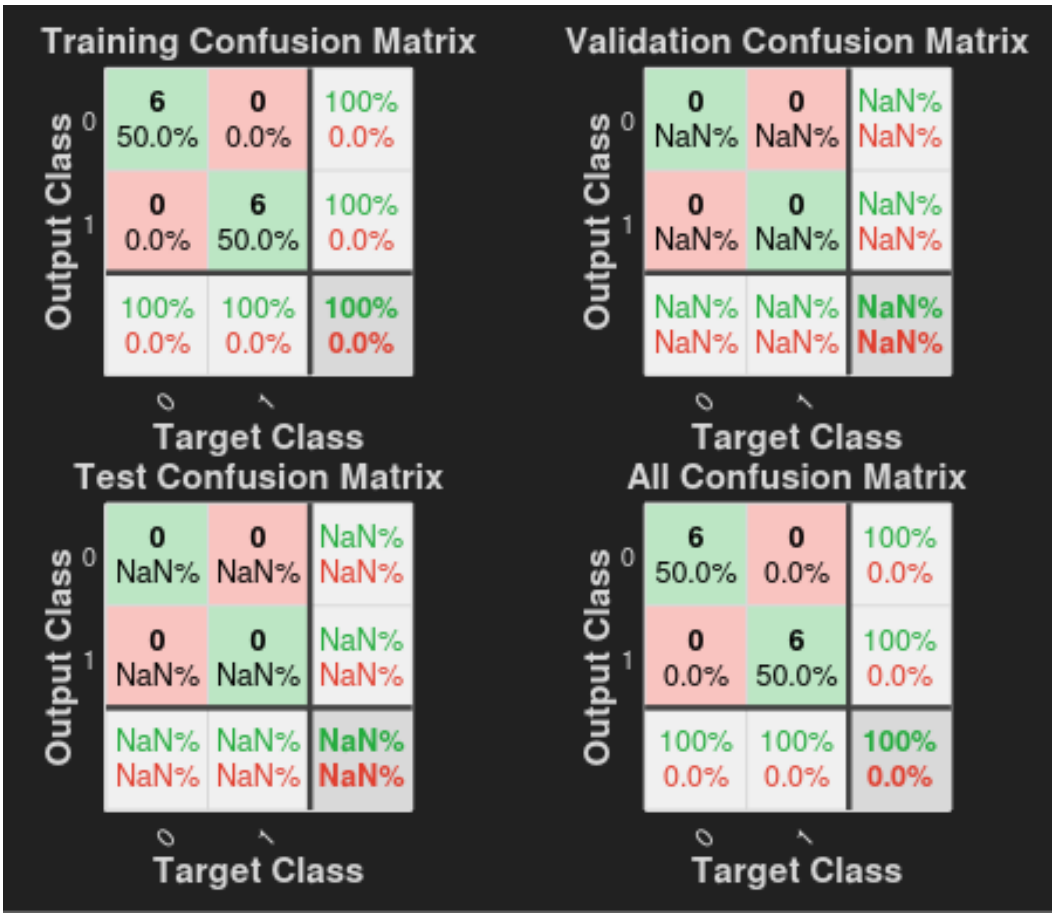
```
P = [2 3 4 5 1 3 6 7 8 9 10 6;
     100 150 180 200 120 170 220 250 300 280 350 210];

T = [0 0 0 0 0 0 1 1 1 1 1 1];

net = perceptron;
net.numInputs = 1;
net.inputs{1}.size = 2;
net.layers{1}.size = 1;
net.layers{1}.transferFcn = 'hardlim';
net_train = train(net, P, T);

Y = sim(net_train, P);

disp('Результати:');
disp(Y);

gscatter(P(1,:), P(2,:), T, ['r','g'], 'ox', 10);
hold on;

plotpc(net_train.IW{1,1}, net_train.b{1});
h = findobj(gca, 'Type', 'line');
set(h, 'LineWidth', 3);
```

```
Результати:
     0     0     0     0     0     0     1     1     1     1     1     1
```

2. Побудувати графік, який зображує простір входів та лінії, що відображають, як налаштована нейронна мережа ділить простір входів на підпростори, які відповідають різним класам.

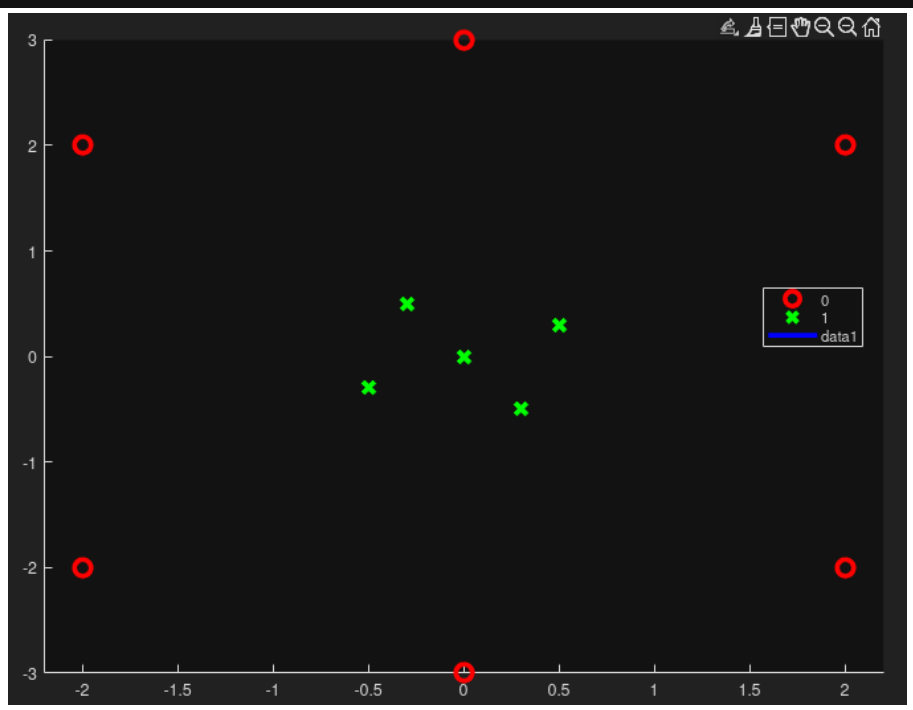

350
300
250
200
150
100
1
2
3
4
5
6
7
8
9
10
0
1
data1


Network Diagram
Training Results
Training finished: Met performance criterion
Training Progress
Unit | Initial Value | Stopped Value | Target Value
Epoch | 0 | 253 | 1000
Elapsed Time | - | 00:00:01 | -
Performance | 0.5 | 0 | 0
Training Algorithms
Data Division: Training Only dividetrain
Training: Cyclical Weight/Bias Rule trainc
Performance: Mean Absolute Error mae
Calculations: MATLAB
Training Plots
Performance
Training State
Confusion
Input
2
Hard Limit
W
b
1
Output
1



3. Вибрати архітектуру та сформувати штучну нейронну мережу для вирішення задачі класифікації двоелементних вхідних векторів відповідно до рис. 2.1.Масштаб графіка вибрати самостійно.

```
P1 = [0 0.5 -0.5 0.3 -0.3; 0 0.3 -0.3 -0.5 0.5];
P2 = [2 -2 2 -2 0 0; 2 2 -2 -2 3 -3];
P = [P1 P2];
T = [1 1 1 1 1 0 0 0 0 0 0];

net = perceptron;
net.numInputs = 1;
net.inputs{1}.size = 2;
net.layers{1}.size = 1;
net.layers{1}.transferFcn = 'hardlim';
net_train = train(net, P, T);

Y = sim(net_train, P);

disp('Результати:');
disp(Y);

gscatter(P(1,:), P(2,:), T, ['r', 'g'], 'ox', 10);
hold on;

plotpc(net_train.IW{1}, net_train.b{1});
h = findobj(gca, 'Type', 'line');
set(h, 'LineWidth', 3);
```



Network Diagram

Training Results

Training finished: Reached maximum number of epochs

Training Progress

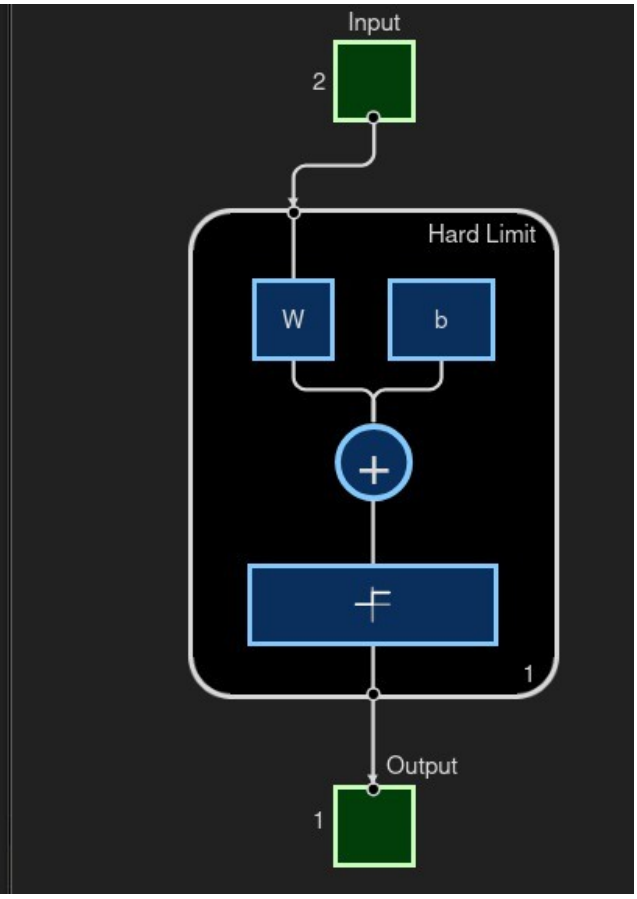
Unit	Initial Value	Stopped Value	Target Value
Epoch	0	1000	1000
Elapsed Time	-	00:00:02	-
Performance	0.545	0.455	0

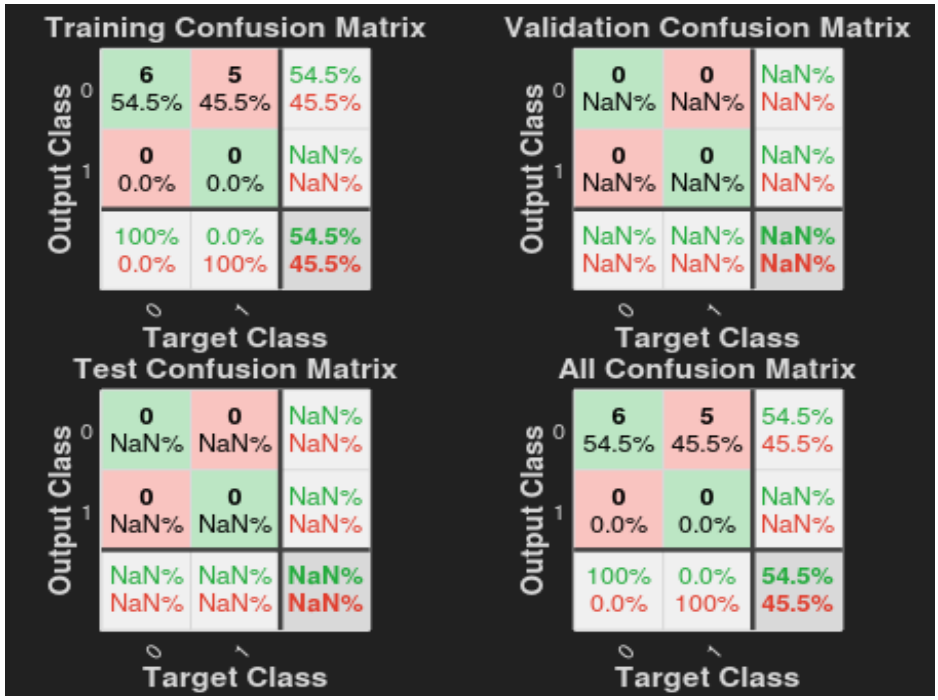
Training Algorithms

Data Division: Training Only dividetrain
Training: Cyclical Weight/Bias Rule trainc
Performance: Mean Absolute Error mae
Calculations: MATLAB

Training Plots

Performance | Training State | Confusion





4. Перевірити працездатність навченої мережі за допомогою моделювання.

```
Результати:
     0     0     0     0     0     0     0     0     0     0     0
```

Контрольні питання

1. В яких випадках для навчання персептрона доцільно застосовувати нормовану функцію навчання?

Нормовану функцію навчання персептрона доцільно застосовувати тоді, коли виникають проблеми зі стабільністю або швидкістю навчання через масштаб вхідних даних.

2. Чи впливає на результат навчання персептрона послідовність пред'явлення навчальних шаблонів?

Так, послідовність пред'явлення навчальних шаблонів впливає на результат навчання персептрона. Персептрон оновлює ваги після кожного навчального прикладу. Тому кожний новий приклад змінює ваги, і наступні приклади обробляються вже з іншими значеннями ваг. Через це різна послідовність подачі шаблонів може призвести до різних траєкторій навчання.

3. Чи можна добитися розділення вхідних образів відповідно до рис. 2.1 за допомогою персептрона, що містить нейрони з нульовими значеннями зміщень?

Ні. Образи на рисунку є лінійно нероздільними, тому персептрон, який реалізує лише лінійну розділяючу поверхню, не може їх розділити. Особливо це неможливо при нульових значеннях зміщень.

4. Яка активаційна функція використовується для нейронів, що утворюють персептрон?

Для нейронів, що утворюють персептрон, зазвичай використовується лінійна активаційна функція або степенева функція активації. У випадку простого персептрону вхідний сигнал підсумовується, і якщо результат перевищує певний поріг, нейрон "активується", в іншому випадку він не активується.

5. Яка кількість нейронів, об'єднаних в одношаровий персептрон, знадобиться, щоб розділити вхідні образи на 10 класів?

Для одношарового персептрону, щоб розділити вхідні образи на 10 класів, вам знадобиться 10 нейронів у вихідному шарі. Кожен нейрон буде відповідати одному з класів.

6. Чи може одношаровий персептрон реалізувати логічну операцію XOR («виключає АБО»)? Обгрунтуйте свою відповідь.

Одношаровий персептрон неможливо навчити для реалізації XOR, оскільки ця операція є нелінійною і не може бути розділена лінійною гіперплощиною. Для досягнення цієї мети необхідно використовувати багатошарові архітектури, які можуть навчатися складнішим концепціям завдяки своїй складній структурі.

Висновок з лабороторної роботи №3:

У ході лабораторної роботи було ознайомлено з основними етапами створення та навчання одношарових персептронів у середовищі Neural Network Toolbox. Було досліджено процес формування архітектури нейронної мережі, ініціалізації ваг та зміщень, а також підготовки вхідних даних у різних форматах для навчання. Особлива увага приділялася способам подачі даних у вигляді числових масивів та послідовностей, що впливає на роботу мережі та її здатність до класифікації. Під час виконання практичних завдань було реалізовано навчання персептрона як послідовним методом за допомогою функції adapt, так і ітеративним із використанням функції train. Це дозволило наочно спостерігати процес корекції ваг та зміщень нейронів, аналізувати динаміку помилки та оцінювати успішність навчання на різних наборах вхідних векторів. Крім того, було показано, що одноразове навчання не гарантує коректної класифікації нових вхідних даних, що підкреслює важливість ітеративного підходу. У завершальній частині роботи було проведено тестування навченого персептрона з використанням функції sim та візуалізацію результатів за допомогою plotvec, plotpv та plotpc. Це дозволило оцінити правильність класифікації вхідних векторів та наочно побачити межу розділення класів у двовимірному просторі. Виконана лабораторна робота дала практичне розуміння принципів побудови і навчання одношарових нейронних мереж та методів контролю їх ефективності.

GitHub: https://github.com/sunset-byte/university_neural_networks_labs